

Module 36: Capstone Project 3

Introduction to C++ Programming — Part 3: Object-Oriented Programming with C++

Module Overview

The final module brings all of Part 3 together. Students design, build, test and document a complete multi-file object-oriented system in their chosen track - the MakersPlace Academy Management System, the Smart Home Energy Management System, or the Hospital Ward Patient Monitoring System - which they have been growing since Module 20. The capstone runs in four committed stages: design and domain model; hierarchies, ownership and resources; behaviour, robustness and persistence; and quality (tests, sanitizers and reflection). Instructors act as design reviewers rather than lecturers, and the deliverables include written design, pattern, testing and reflection documents as well as code.

Suggested Duration: 4 sessions of 60–90 minutes each (one per stage), plus independent lab/studio time

Learning Objectives

By the end of this module, students will be able to:

- Design an object-oriented system from a problem statement and document it with UML, CRC cards and an ownership table.
- Organise a multi-file CMake project in a namespace, with an application and a separate test program.
- Implement polymorphic hierarchies and interfaces with clear smart-pointer ownership and no manual memory management.
- Build a complete value type, an exception hierarchy, a strong-guarantee batch operation and concept-constrained templates.
- Apply at least five design patterns where they genuinely solve a problem, and document them.
- Persist data across runs with robust, error-translating loading.
- Prove correctness with 40+ automated checks and sanitizer-clean builds, and reflect critically on the design using SOLID.

Prerequisites

- Modules 20-35, including each module's track project - the capstone extends that work.
- Part 2 Modules 14 and 16-17 (file I/O, STL containers and algorithms).

Materials & Instructor Prep

- 36-capstone-project-3/README.md (the four stages and three track specifications) and each track's starter scaffold.
- A grading rubric derived from the README's concept map and 'You're done when...' list - share it with students at the start.

- Time in the schedule for one short design review per student (or team) at the end of Stage 1.

Key Vocabulary

Term	Definition
Scaffold	A starter project that provides structure (build files, folders, key headers) but not the solution.
Ownership table	A table listing every pointer-like member and whether it owns, shares, observes or borrows, with the reason.
Demo mode	A <code>--demo</code> command-line option that runs a scripted scenario with no keyboard input, so anyone can see the system work.
Design review	A short structured conversation in which a reviewer questions a design before (or while) it is built.
Definition of done	The explicit list of conditions a piece of work must meet before it counts as finished.

Lesson Outline & Instructor Talking Points

Session 1 - Stage 1: Design and domain model (Modules 20-22, 27)

- Walk through the three track specifications and the four stages. Share the rubric and the definition of done.
- Students write `DESIGN.md` before any code. Circulate and run short design reviews: ask 'who owns this?', 'what's the invariant?', and 'is this really is-a?'
- Students set up the CMake scaffold, namespace and core entity classes, and commit Stage 1.

Instructor note: Do not let students skip `DESIGN.md`. Ten minutes of review here prevents most of the problems you would otherwise mark at the end.

Session 2 - Stage 2: Hierarchies, ownership and resources (Modules 23-26, 28-29)

- Students implement the abstract base and interfaces, store polymorphic objects in containers of `unique_ptr`, and write their RAI class.
- Run a whole-class grep for new and delete, and a quick review of each ownership table against the code.

Instructor note: Watch for `shared_ptr` used 'to be safe' and for slicing through `by-value` containers - both are common at this stage.

Session 3 - Stage 3: Behaviour, robustness and persistence (Modules 30-32, 34)

- Students complete the value type's operators, exception hierarchy, strong-guarantee batch operation, `Repository<T>` and `Statistics<T>`, patterns, persistence and the `--demo` scenario.
- Pair students to run each other's `--demo` and to try corrupting each other's data files.

Instructor note: *This is the densest stage. It is reasonable to split it across two sessions: behaviour and patterns first, persistence and demo second.*

Session 4 - Stage 4: Quality (Modules 33, 35)

- Students finish the test suite (40+ checks with fakes), run tests and demo under sanitizers, and write REFLECTION.md and the track README.
- Close with short presentations: each student demonstrates their system and explains one design decision and one thing they would change.

Instructor note: *Protect the presentations even under time pressure - explaining a design aloud is a professional skill this course has been building towards.*

Common Student Misconceptions

Misconception	How to Address It
"The capstone is mostly about getting the program to run."	Working code is necessary but not sufficient: design, ownership, robustness, tests and reflection are all graded.
"Using more patterns earns more marks."	Five well-justified patterns beat ten forced ones. PATTERNS.md must show the problem each solves - and one place a pattern was deliberately not used.
"Tests can be written at the end."	Tests written during Stages 1-3 catch bugs early and make later stages safer; leaving them to Stage 4 makes the work harder.
"REFLECTION.md is a formality."	It is where students show they understand why their design is shaped as it is. Generic statements score poorly; specific examples from their code score well.

Exercises & Module Project

The capstone itself IS the assignment - no separate exercises. See 36-capstone-project-3/README.md for the four stages and the three tracks: Generic - MakersPlace Academy Management System; EE - Smart Home Energy Management System; Biomedical - Hospital Ward Patient Monitoring System. Each track's starter scaffold builds an app with a --demo mode and a test program run by ctest.

Facilitation guidance: Grade holistically against the concept map, the four stages and the 'You're done when...' list rather than a single correct output. Value clear ownership, honest exception guarantees, justified patterns, a test suite that would catch real bugs, and a reflection that cites specific code. Build every submission yourself with -fsanitize=address,undefined and run both --demo and ctest.

Discussion & Comprehension Check Questions

1. Which relationship in your design was hardest to classify (composition, aggregation or association), and how did you decide?

Guidance: Look for lifetime-based reasoning: who creates and destroys the part, and whether it can exist independently or belong to several wholes.

2. Where does your system give the strong guarantee, and how do you know it works?

Guidance: Expect a specific batch operation, a description of validating before committing, and a test that triggers a failure and checks the state is unchanged.

3. Which pattern made a later change easiest, and what was the change?

Guidance: Strong answers name a concrete change - a new device type, grading scheme or alert channel - and show it was added rather than edited in.

Assessment Ideas

- Stage-by-stage commit review: one commit (or more) per stage, with messages that describe the work.
- Final demonstration and short design defence: run --demo and ctest live, then answer two questions about ownership and one about a pattern.
- Written deliverables: DESIGN.md, PATTERNS.md, TESTING.md, REFLECTION.md and the track README, marked for specificity and accuracy against the code.

Answer Key Notes

There is no single answer key: every design will differ. Use the README's concept map and 'You're done when...' list as the rubric, and the track starters' scaffolds as the expected structure.

Wrap-Up & Bridge to Next Module

Close the whole course here: "From Hello, World! to a designed, tested, multi-file object-oriented system. Point students to the 'Where to go from here' list - concurrency, the modern standard library, embedded C++ and robotics, graphical and networked apps - and remind them that the habits matter more than any single topic."

Instructor Tips

- Schedule design reviews at the end of Stage 1; they have the largest effect on final quality.
- Encourage many small commits within each stage, not just one per stage.
- If students are working in teams, require each member to own at least one class end to end, including its tests.